PSTAKE

SMART CONTRACT AUDIT

ZOKYO.

May 25, 2021 | v. 1.0

PASS

Zokyo's Security Team has concluded that this smart contract passes security qualifications to be listed on digital asset exchanges

TECHNICAL SUMMARY

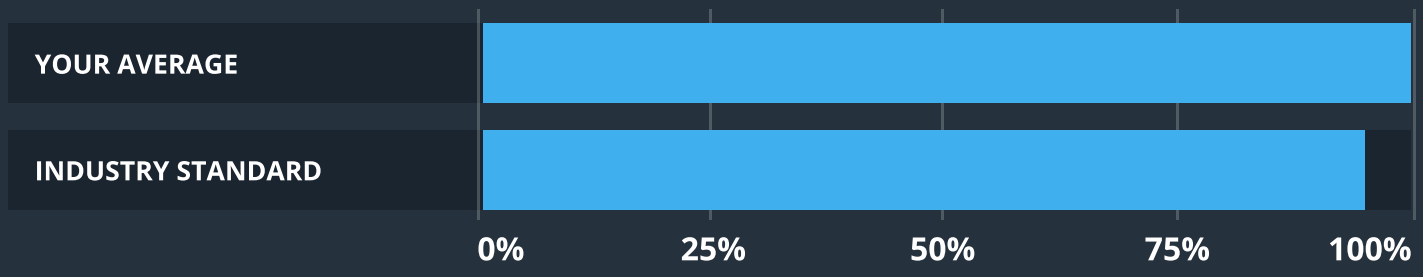
This document outlines the overall security of the pStake smart contracts, evaluated by Zokyo's Blockchain Security team.

The scope of this audit was to analyze and document the pStake smart contract codebase for quality, security, and correctness.

Contract Status

There were no critical issues found during the audit.

Testable Code



Testable code is 100% which is above the industry standard of 95%.

It should be noted that this audit is not an endorsement of the reliability or effectiveness of the contract, rather limited to an assessment of the logic and implementation. In order to ensure a secure contract that's able to withstand the Ethereum network's fast-paced and rapidly changing environment, we at Zokyo recommend that the pStake team put in place a bug bounty program to encourage further and active analysis of the smart contract.

TABLE OF CONTENTS

Auditing Strategy and Techniques Applied 3
Executive Summary 4
Structure and Organization of Document 5
Complete Analysis 6
Code Coverage and Test Results for all files 16
Tests written by Zokyo Secured team 16

AUDITING STRATEGY AND TECHNIQUES APPLIED

The Smart contract's source code was taken from the pStake repository – https://github.com/persistenceOne/pStakeSmartContracts/.
Last commit – 1a7c33b5d29117c57322dce4970bd2f0020ca974.

Throughout the review process, care was taken to ensure that the token contract:

- Implements and adheres to existing Token standards appropriately and effectively;
- Documentation and code comments match logic and behavior;
- Distributes tokens in a manner that matches calculations;
- Follows best practices in efficient use of gas, without unnecessary waste;
- Uses methods safe from reentrance attacks;
- Is not affected by the latest vulnerabilities;
- Whether the code meets best practices in code readability, etc.

Zokyo's Security Team has followed best practices and industry-standard techniques to verify the implementation of pStake smart contracts. To do so, the code is reviewed line-by-line by our smart contract developers, documenting any issues as they are discovered. Part of this work includes writing a unit test suite using the Truffle testing framework. In summary, our strategies consist largely of manual collaboration between multiple team members at each stage of the review:

1	Due diligence in assessing the overall code quality of the codebase.	3	Testing contract logic against common and uncommon attack vectors.
2	Cross-comparison with other, similar smart contracts by industry leaders.	4	Thorough, manual review of the codebase, line-by-line.

EXECUTIVE SUMMARY

There were no critical issues found during the audit. All the mentioned findings may have an effect only in case of specific conditions performed by the contract owner.

The overall smart contracts' code is well written and structured. There were only Medium, Low and Informational bugs found during the audit. Despite the code style is not ideal, the code's readability is decent.

The findings during the audit have no impact on contract performance, so it is fully production-ready.

STRUCTURE AND ORGANIZATION OF DOCUMENT

For ease of navigation, sections are arranged from most critical to least critical. Issues are tagged “Resolved” or “Unresolved” depending on whether they have been fixed or addressed. Furthermore, the severity of each issue is written as assessed by the risk of exploitation or other unexpected or otherwise unsafe behavior:

Critical

The issue affects the ability of the contract to compile or operate in a significant way.

High

The issue affects the ability of the contract to compile or operate in a significant way.

Medium

The issue affects the ability of the contract to operate in a way that doesn’t significantly hinder its behavior.

Low

The issue has minimal impact on the contract’s ability to operate.

Informational

The issue has no impact on the contract’s ability to operate.

COMPLETE ANALYSIS

UToken.sol

INFORMATIONAL | UNRESOLVED

The unmatching comment. The documentary comment for the setLiquidStakingContract() function does not match its signature.

UToken.sol

LOW | UNRESOLVED

Useless parameter variable:

```
function withdrawUTokens(address from, uint256 tokens, string memory
toAtomAddress) public virtual override whenNotPaused {
    require(tokens>0, "TokenWrapper: Number of unstaked tokens should
    be greater than 0");
    uint256 _currentUTokenBalance = _uTokens.balanceOf(from);
    require(_currentUTokenBalance>=tokens, "TokenWrapper: Insuffcient
    balance for account");
    require(from == _msgSender(), "TokenWrapper: Withdraw can only be done by Staker");
    _uTokens.burn(from, tokens);
    emit WithdrawUTokens(from, tokens, toAtomAddress,
    block.timestamp);
}
```

Recommendation:
The from variable can be omitted and _msgSender() used instead.

SToken.sol

MEDIUM | UNRESOLVED

The useless if and SafeMath omission:

```
function calculatePendingRewards(address to) public view virtual override whenNotPaused returns (uint256
pendingRewards){
    // Get the current Block
    uint256 _currentBlock = block.number;
    // Get the time in number of blocks
    uint256 _rewardBlock = _currentBlock.sub(_stakedBlocks[to], "STokens: Error in subtraction");
    // Get the balance of the account
    uint256 _balance = balanceOf(to);
    // Calculate the interest if P, R, T are non zero values
    if(_balance > 0 && _rewardRate > 0 && _rewardBlock > 0) {
        pendingRewards = (_balance * _rewardRate * _rewardBlock) / 100;
    }
    return pendingRewards;
}
```

The if statement is unnecessary here and the omission of SafeMath may lead to the overflow.

Recommendation:
Remove the if statement and implement proper SafeMath calls.

LOW | UNRESOLVED

The _wrapperContract is not used:

```
address private _wrapperContract;
```

Recommendation:
Remove this variable and the corresponding setter.

LOW | UNRESOLVED

The improper usage of return statement:

```
function getRewardRate() public view virtual override whenNotPaused returns (uint256 rewardRate) {
        rewardRate = _rewardRate;
    return rewardRate;
}
```

Recommendation:
The return statement can be safely removed.

LOW | UNRESOLVED

The improper usage of return statement:

```
function getStakedBlock(address to) public view virtual override whenNotPaused returns (uint256
stakedBlocks) {
    stakedBlocks = _stakedBlocks[to];
    return stakedBlocks;
}
```

Recommendation:
The return statement can be safely removed.

LOW | UNRESOLVED

Useless parameter variable.

```
function calculateRewards(address to) public virtual override whenNotPaused returns (bool success) {
    require(to == _msgSender(), "STokens: only staker can initiate their own rewards calculation");
    uint256 reward =  _calculateRewards(to);
    emit TriggeredCalculateRewards(to, reward, block.timestamp);
    return true;
}
```

Recommendation:
The to variable in calculateRewards() can be omitted and _msgSender() used instead.

LiquidStaking.sol

MEDIUM | UNRESOLVED

SafeMath omission:

```
function withdrawUnstakedTokens(address staker) public virtual override whenNotPaused{
    ...
    for (uint256 i=0; i<_unstakingExpiration[staker].length; i++) {
        if (block.timestamp > _unstakingExpiration[staker][i]) {
            _withdrawBalance = _withdrawBalance +
            _unstakingAmount[staker][i];
            ...
        }
    }
}
```

Recommendation:
Add proper SafeMath function.

MEDIUM | UNRESOLVED

Evergrowing array:

```
function withdrawUnstakedTokens(address staker) public virtual override whenNotPaused{
    require(staker == _msgSender(), "LiquidStaking: Only staker can withdraw");
    // require(hasRole(STAKER_ROLE, _msgSender()), "LiquidStaking: Only staker can withdrawr");
    uint256 _withdrawBalance;
    for (uint256 i=0; i<_unstakingExpiration[staker].length; i++) {
        if (block.timestamp > _unstakingExpiration[staker][i]) {
            _withdrawBalance =_withdrawBalance +
            _unstakingAmount[staker][i];
            _unstakingExpiration[staker][i] = 0;
            _unstakingAmount[staker][i] = 0;
        }
    }
    ...
}
```

The problem with this loop is that on every unStake() the user pushes a new unstake value to the array increasing its length, but on withdrawUnstakedTokens() array's length is not decreased. If the user repeats the process of (stake -> unstake -> withdraw) some amount of times, he would no longer be able to withdraw the tokens.

Recommendation:
Because the array grows only to its right, it would be helpful to have a counter variable indicating from what index to start the loop. More precisely:

```
mapping(address => uint256) internal _counters;
function withdrawUnstakedTokens() {
    ...
    for (uint256 i = _counters[_msgSender()]; i < _unstakingExpiration[staker].length; i++) {
        if (block.timestamp > _unstakingExpiration[staker][i]) {
            _withdrawBalance =_withdrawBalance +
            _unstakingAmount[staker][i];
            _unstakingExpiration[staker][i] = 0;
            _unstakingAmount[staker][i] = 0;
```

```
            _counters[_msgSender()]++;
        }
    }
    ...
}
```

LOW | UNRESOLVED

Unspecified visibility of variables:

```
uint256 _unstakinglockTime;

//Mapping to handle the Expiry period
mapping(address => uint256[]) _unstakingExpiration;

//Mapping to handle the Expiry amount
mapping(address => uint256[]) _unstakingAmount;
```

Recommendation:
Explicitly specify the visibility.

LOW | UNRESOLVED

Useless parameter variable:

```
function stake(address to, uint256 utok) public virtual override whenNotPaused returns(bool)  {
    // Check the supplied amount is greater than 0
    require(utok>0, "LiquidStaking: Number of staked tokens should be greater than 0");
    require(to == _msgSender(), "LiquidStaking: Staking can only be done by Staker");
    ...
}
```

Recommendation:
The to variable can be omitted and _msgSender() used instead.

LOW | UNRESOLVED

The _tokenWrapper is not used:

```
address private _tokenWrapper;
```

Recommendation:
Remove this variable and the corresponding setter.

LOW | UNRESOLVED

Useless parameter variable:

```
function unStake(address to, uint256 stok) public virtual override whenNotPaused returns(bool) {
    // Check the supplied amount is greater than 0
    require(to == _msgSender(), "LiquidStaking: Unstaking can only be done by Stakerr");
    ...
}
```

Recommendation:
The to variable can be omitted and _msgSender() used instead.

LOW | UNRESOLVED

Useless parameter variable:

```
function withdrawUnstakedTokens(address staker) public virtual override whenNotPaused{
    require(staker == _msgSender(), "LiquidStaking: Only staker can withdraw");
    ...
}
```

Recommendation:
The staker variable can be omitted and _msgSender() used instead.

LOW | UNRESOLVED

The direct storage read:

```
function withdrawUnstakedTokens(address staker) public virtual override whenNotPaused{
    require(staker == _msgSender(), "LiquidStaking: Only staker can withdraw");
    // require(hasRole(STAKER_ROLE, _msgSender()), "LiquidStaking: Only staker can withdrawr");
    uint256 _withdrawBalance;
    for (uint256 i=0; i<_unstakingExpiration[staker].length; i++)
    ...
}
```

Recommendation:
Because this length value is read on every single loop iteration, it consumes a lot of gas, caching it into a memory variable would save the transaction cost.

LOW | UNRESOLVED

Useless parameter variable:

```
function getTotalUnbondedTokens(address staker) public view virtual override whenNotPaused returns
(uint256 unbondingTokens) {
    if(staker == _msgSender()){
    ...
    }
}
```

Recommendation:
The staker variable can be omitted and _msgSender() used instead.

LOW | UNRESOLVED

SafeMath omission:

```
function getTotalUnbondedTokens(address staker) public view virtual override whenNotPaused returns
(uint256 unbondingTokens) {
    ...
    unbondingTokens = unbondingTokens +
    _unstakingAmount[staker][i];
    ...
}
```

Recommendation:
Add proper SafeMath function.

INFORMATIONAL | UNRESOLVED

The direct storage read:

```
function getTotalUnbondedTokens(address staker) public view virtual override whenNotPaused returns
(uint256 unbondingTokens) {
    ...
    for (uint256 i=0; i<_unstakingExpiration[staker].length; i++) {
        if (block.timestamp > _unstakingExpiration[staker][i]) {
            unbondingTokens = unbondingTokens +
            _unstakingAmount[staker][i];
        }
    }
}
```

Recommendation:
Because this length value is read on every single loop iteration, it consumes a lot of gas, caching it into a memory variable would save the transaction cost.

Other recommendations

https://docs.soliditylang.org/en/v0.5.3/style-guide.html
https://docs.soliditylang.org/en/v0.6.2/security-considerations.html#tx-origin

Appendix

There is a tiny problem with the SToken concept. For instance, if one wanted to create a Uniswap SToken to Ether pair, he would need to provide liquidity into this pool e.i. both SToken and Ether. However, providing SToken is prone to not receiving staking rewards, because the tokens would be held by the Uniswap pool contract. As a result, a liquidity provider is not incentivized to provide liquidity. Moreover, this might bring a lot of UTokens that are locked forever.

CODE COVERAGE AND TEST RESULTS FOR ALL FILES

Tests written by Zokyo Secured team

As part of our work assisting pStake in verifying the correctness of their contract code, our team was responsible for writing integration tests using the Truffle testing framework.

Tests were based on the functionality of the code, as well as a review of the pStake contract requirements for details about issuance amounts and how the system handles these.

```
Contract: TokenWrapper
  TokenWrapper Test Cases
  ✓ should set uToken contract only by bridge owne (1006ms)
  ✓ should set sToken contract only by bridge owner (405ms)
  ✓ should set LiquidStaking contract only by bridge owner (387ms)
  ✓ should pause contract correctly (589ms)
  ✓ should unpause contract correctly (1487ms)
  ✓ should generate uTokens correctly (801ms)
  ✓ should withdraw uTokens correctly (1859ms)

Contract: LiquidStaking
  LiquidStaking Test Cases
  ✓ should set uToken contract only by bridge owner (813ms)
  ✓ should set sToken contract only by bridge owner (316ms)
  ✓ should set Wrapper contract only by bridge owner (474ms)
  ✓ should stake correctly (2018ms)
  ✓ should unstake correctly (2566ms)
  ✓ should withdraw unstaked tokens correctly (3056ms)
  ✓ should pause contract correctly (666ms)
  ✓ should unpause contract correctly (897ms)

Contract: STokens
  STokens Test Cases
  ✓ should deploy with correct name (278ms)
  ✓ should deploy with correct symbol (307ms)
  ✓ should deploy with correct decimals (441ms)
```

```
    ✓ should set uToken contract only by bridge owner (1079ms)
    ✓ should set reward rate correctly (3271ms)
    ✓ should set LiquidStaking contract only by bridge owner (523ms)
    ✓ should set Wrapper contract only by bridge owner (665ms)
    ✓ should pause contract correctly (630ms)
    ✓ should unpause contract correctly (986ms)
    ✓ should mint tokens correctly (2082ms)
    ✓ should burn tokens correctly (2835ms)
    ✓ should calculate pending rewards correctly (2766ms)

  Contract: UTokens
    UTokens Test Cases
    ✓ should deploy with correct name (151ms)
    ✓ should deploy with correct symbol (109ms)
    ✓ should deploy with correct decimals (160ms)
    ✓ should set sToken contract only by bridge owner (936ms)
    ✓ should set LiquidStaking contract only by bridge owner (504ms)
    ✓ should set Wrapper contract only by bridge owner (521ms)
    ✓ should pause contract correctly (680ms)
    ✓ should unpause contract correctly (1103ms)
    ✓ should mint tokens correctly (1047ms)
    ✓ should burn tokens correctly (1416ms)

  37 passing (6s)
```

FILE	% STMTS	% BRANCH	% FUNCS	% LINES
contracts/	100.00	97.92	100.00	100.00
LiquidStaking.sol	100.00	96.88	100.00	100.00
STokens.sol	100.00	96.67	100.00	100.00
TokenWrapper.sol	100.00	100.00	100.00	100.00
UTokens.sol	100.00	100.00	100.00	100.00
contracts/interfaces/	100.00	100.00	100.00	100.00
ILiquidStaking.sol	100.00	100.00	100.00	100.00
ISTokens.sol	100.00	100.00	100.00	100.00
ITokenWrapper.sol	100.00	100.00	100.00	100.00
IUTokens.sol	100.00	100.00	100.00	100.00
All files	**96.99**	**97.92**	**100.00**	**100.00**

We are grateful to have been given the opportunity to work with the pStake team.

The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

Zokyo's Security Team recommends that the pStake team put in place a bug bounty program to encourage further analysis of the smart contract by third parties.

ZOKYO.